Playing with the Rubik's Cube and its God's number in Rocq

Laurent Théry

INRIA, Stamp Team
Laurent.Thery@inria.fr

Abstract. God's number, the largest number of face turns needed to solve a Rubik's cube, is twenty. It was settled in 2010 by a computation of thirty-five processor years. Even though this kind of computation cannot easily be replicated in a proof assistant like Rocq, three smaller results about the Rubik's cube are proved here. First, we prove that twenty is a lower bound: one position, the superflip, cannot be solved in nineteen. A half turn can also count as two moves. The number is then twenty-six. We prove that twenty-six is a lower bound for solving the four-spot with the superflip on it. This is our second result. Finally, the published computation splits the cube into the 2 217 093 120 cosets of a subgroup, one search to a coset. A coset contains 19 508 428 800 positions. We formalise the correctness of a coset search, and we apply it to one specific coset. It follows that every position of the superflip's coset is solved in twenty moves or fewer. This is our last result.

Keywords. Rubik's cube, God's number, formal proof, Rocq, group theory.

1 The problem

A Rubik's cube is built from twenty-six small cubes: **eight corner pieces** with three stickers each, **twelve edge pieces** with two, and **six centre pieces** with one. The centres are attached to the core. They spin in place but never travel, so they fix the frame. The white face is wherever the white centre is. A face turn moves four corners and four edges, and nothing else.

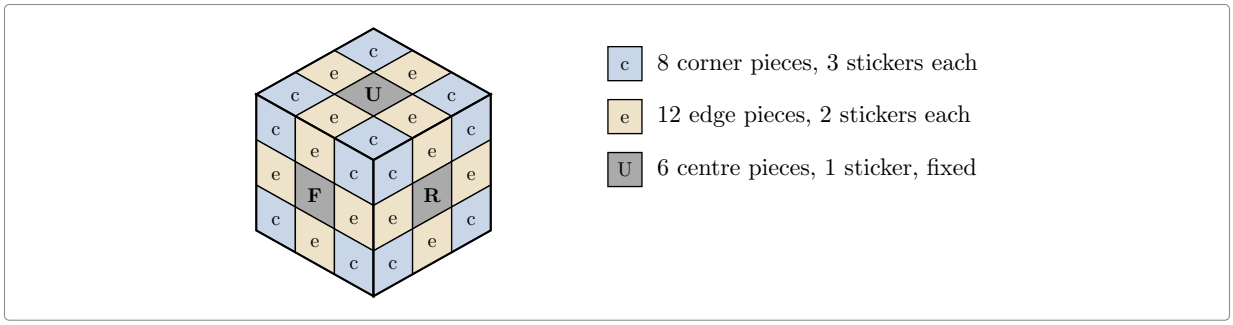


Figure 1: The three kinds of piece. Every face shows four corner stickers, four edge stickers and one centre.

Not every arrangement of the pieces can be reached by turning faces. The number of arrangements that can be reached is

$$8! \cdot 3^7 \cdot 12! \cdot 2^{11}/2 = 43\ 252\ 003\ 274\ 489\ 856\ 000 \approx 4.3 \cdot 10^{19},$$

which reads: the eight corners in any order (8!), each twisted one of three ways except that the last is forced by the other seven (3^7); the twelve edges in any order (12!), each flipped or not with the last again forced (2^{11}); and a final halving, because corners and edges cannot be rearranged independently of each other.

Turning one face is a *move*, and a half turn counts as one move just like a quarter turn. Every scramble can be undone. The question is how many moves the worst scramble needs. That number is called **God's number**.

Counting a half turn as one move is a choice. A half turn can also count as two moves, and that gives a second number for the same cube. We prove a lower bound for each.

In 2010 Rokicki, Kociemba, Davidson and Dethridge showed that it is **20** [1]. Their result has two halves. One half is that twenty moves always suffice. It is the huge one, and the last section of this note says how it was obtained. The other half is that twenty moves are sometimes needed. For that it is enough to take one scramble and show it cannot be solved in 19.

We take one scramble: the **superflip**, drawn in Figure 2 beside a solved cube. Every corner sticker is where it belongs. Every edge is in its own place but turned over, so it shows the colour of the face beside it. Look at the cube from any angle, or in a mirror, and the pattern is the same. The superflip is one of the rare positions that all 48 ways of looking at a cube leave unchanged, and that matters later. A 20-move solution for it is known, so ruling out a 19-move solution puts the superflip at distance exactly 20. That is what we prove.

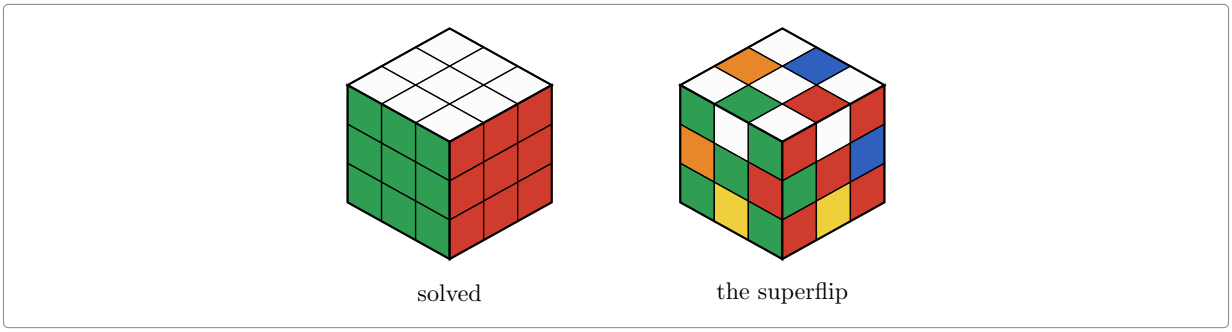


Figure 2: A solved cube, and the superflip.

That is still a big computation. There are 18 moves at each step, so 19 moves means about 18^{19} words.

Three key facts about a position are used in what follows.

- Each corner has exactly one sticker of the top colour or the bottom colour. That sticker can be in three places on the corner. It can be on the top or bottom face, which we write 0, or on one of the corner's two sides, which we write 1 and 2. We call this the corner's **twist**.
- Each edge has a right way round, which we write 0. Put back the other way round, it shows its two colours the wrong way about, and that we write 1. That is what we call the edge's **flip**.
- Four of the twelve edges belong in the middle layer, between the top and the bottom face. We call that layer the **slice**. A move can send those four edges to any of the twelve edge slots. Which four slots they are in is one of 495 choices, and we write 0 for the four slots of the slice itself.

The three are easy to read on the superflip. Every corner is home and the right way up, so every top or bottom sticker is on the top or bottom face and its eight twists are all zero. Every edge is turned over, so its twelve flips are all one. Every edge is also in its own slot, so the four middle edges are back in the four slots of the slice, which is 0.

2 The cube as permutations

The cube is easier to reason about if we stop treating it as a solid object. Only the coloured stickers matter. There are six faces of nine stickers, and the six centre stickers never move relative to each other. So a move is a rearrangement of the **48 remaining stickers**. We number them 0 to 47, as in Figure 3 and Figure 4: eight to a face, taken left to right and top to bottom with the centre skipped, so up gets 0–7, left 8–15, front 16–23, right 24–31, back 32–39 and down 40–47. The sources use these numbers, so a move can be checked against a picture.

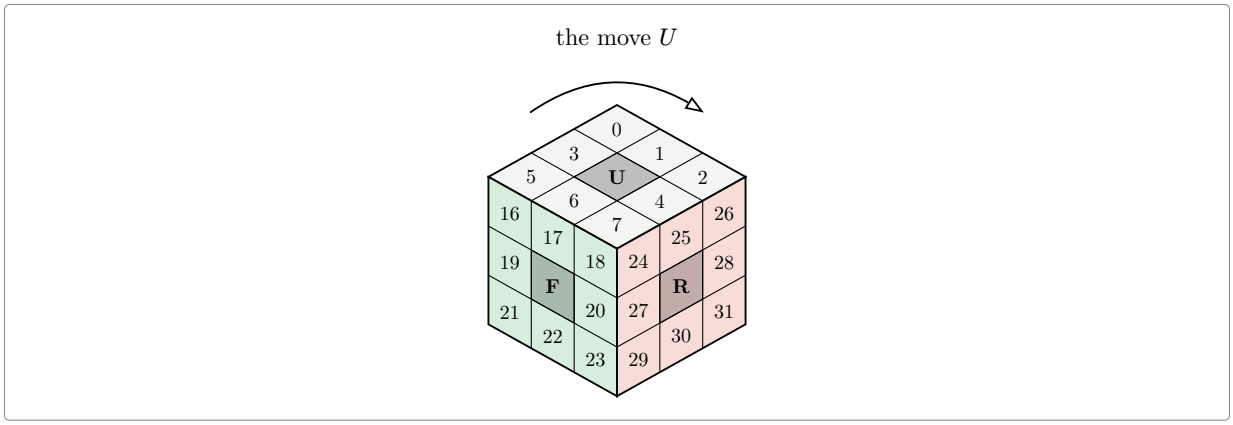


Figure 3: Three of the six faces, and the turn of the top one.

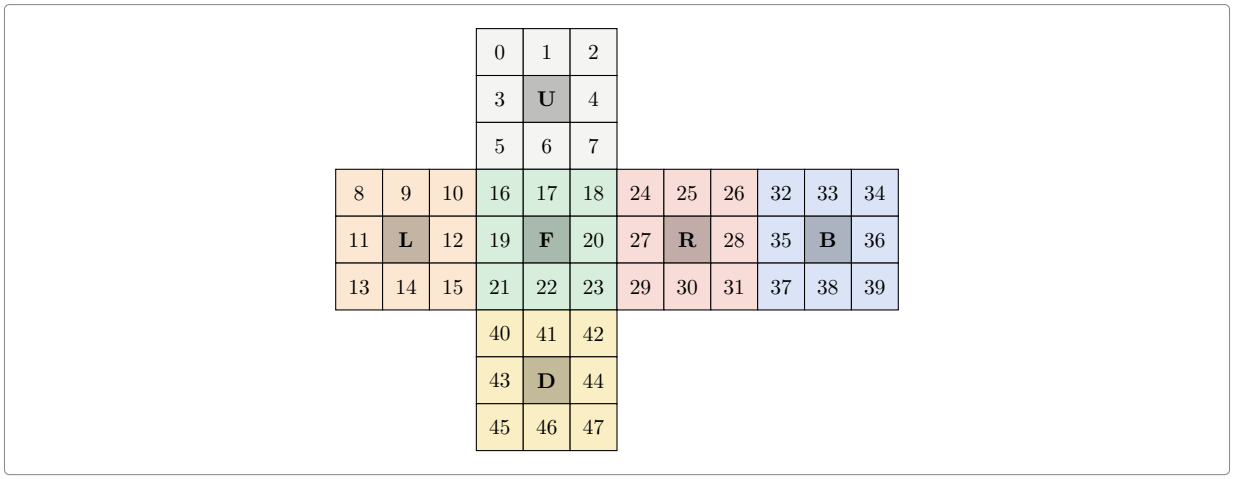


Figure 4: The cube unfolded, with all forty-eight places numbered.

With the places numbered, a move is written down by saying where the sticker in each place goes. Turning the top face clockwise carries the sticker in corner 0 to corner 2, the one in 2 to 7, the one in 7 to 5 and the one in 5 back to 0. That is a four step cycle, written (0 2 7 5). The four edge stickers of that face do the same, (1 4 6 3). The turn does not only move the top face. It also carries the top row of each side face round to the next one: front to left, left to back, back to right, right to front. That is three more cycles, (8 32 24 16) and its two companions. Figure 5 shows all of it, each square saying which sticker sits there afterwards. The top row of the left face holds 16, 17, 18, the stickers that came round from the front. The other forty stickers stay where they are, and the six centres never move.

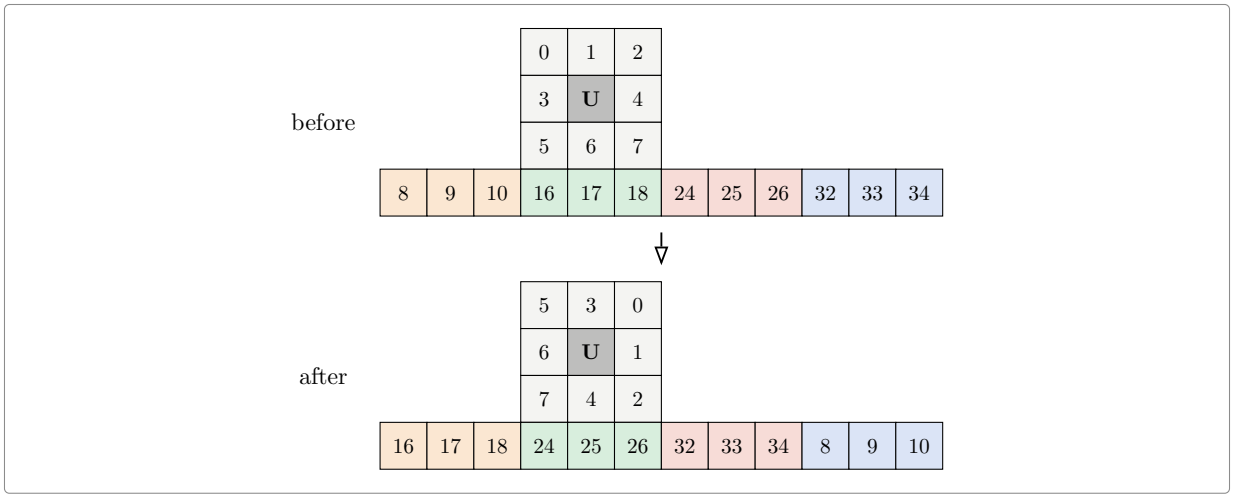


Figure 5: Everything a clockwise turn of the top face moves.

Six clockwise quarter turns generate everything: up, right, front, down, left and back. Each of them can also be done twice or backwards, which gives the **eighteen moves**

$$\begin{array}{cccccc} U & R & F & D & L & B \\ U^2 & R^2 & F^2 & D^2 & L^2 & B^2 \\ U^{-1} & R^{-1} & F^{-1} & D^{-1} & L^{-1} & B^{-1} \end{array}$$

A scramble is a product of moves, for instance $RUR^{-1}U^{-1}$, a word of length 4. The set of all scrambles is a group G: the **cube group**. Solving a scramble in d moves means writing it as a word of d moves. So "solvable in at most d moves" says that the scramble lies in the **ball of radius d** around the solved cube. God's number is the largest distance that occurs.

2.1 The cube in Rocq

The development is written in the Rocq prover [2] and built on **mathcomp** [3], a large library of formalised mathematics that already has permutations, groups and products. The cube file is a transcription of the paragraphs above, and it is short:

```
Definition facelet := 'I_48.

Definition Umove : {perm facelet} :=
  cyc [:: 0@; 2@; 7@; 5@] * cyc [:: 1@; 4@; 6@; 3@] *
  cyc [:: 8@; 32@; 24@; 16@] * cyc [:: 9@; 33@; 25@; 17@] *
  cyc [:: 10@; 34@; 26@; 18@].
  (* ... and five more, one per face ... *)

Definition faces : seq {perm facelet} :=
  [:: Umove; Rmove; Fmove; Dmove; Lmove; Bmove].
Definition moves : seq {perm facelet} :=
  flatten [seq [:: g; g ^+ 2; g ^-1] | g <- faces].
Definition G : {group {perm facelet}} := <<Sset>>.
```

Word by word:

- `'I_48` is the type of the whole numbers **below** 48, so the places are numbered **0 to 47** and not 1 to 48, everywhere in the sources and in the pictures of this note.
- `{perm facelet}` is the type of **permutations** of those places: a way of sending each place to a place, no two of them landing on the same one. That is exactly what a scramble is.
- `cyc [:: 0@; 2@; 7@; 5@]` is the **cycle** that sends 0 to 2, 2 to 7, 7 to 5 and 5 back to 0, leaving the other forty-four places where they are. The `@` is local notation turning a plain number into a place.

- * composes two permutations, so `Umove` is the five cycles of Figure 5 done together, and `g ^+ 2` and `g ^-1` are the same turn done twice and undone.
- `seq` is a list, and `faces` is the list of the six clockwise quarter turns. `moves` runs through it and keeps three moves per face, which is the eighteen.
- `<<Sset>>` is the group generated by a set: everything reachable by composing moves, which is the cube group.

The superflip is written down the same way, as the twelve swaps that exchange the two stickers of each edge:

```
Definition Spcyc : seq (seq facelet) :=
  [:: [:: 1@; 33@]; [:: 3@; 9@]; [:: 4@; 25@];
      (* ... nine more, one per edge ... *) ].

Definition superflip : {perm facelet} := \prod_(l <- Spcyc) cyc l.
```

That makes it a permutation of the stickers, but on its own it says nothing about the cube. A permutation is a legal position only if the faces can be turned to reach it, that is, only if it lies in G. The definition above does not give that. It has to be proved.

The proof is one equality. On the left, the superflip as just defined. On the right, a word of twenty moves:

$$U\ R^2\ F\ B\ R\ B^2\ R\ U^2\ L\ B^2\ R\ U^{-1}\ D^{-1}\ R^2\ F\ R^{-1}\ L\ B^2\ U^2\ F^2$$

Both sides are permutations of the 48 stickers. Each is written out as the list of the 48 places it sends each place to, so the equality is one comparison of two lists. Every letter on the right is one of the eighteen moves, so the superflip lies in G. That same word gives the upper bound of 20 for this one position.

Nothing here is assumed. There is no axiom saying what a cube is. A reader who wants to check the model has only to compare the six lists of cycles against Figure 4. After this file, stickers are never mentioned again.

3 The search in outline

Two classical ideas make the search possible.

3.1 The pruning estimate

Neither idea is new. A depth-first search that deepens step by step and prunes on an estimate that is never too big is Korf's IDA* [4]. Taking the estimate from a table of exact distances in a simplified version of the puzzle is a *pattern database* [5], and Korf solved the cube optimally with three of them [6]. The summary used here is Kociemba's, from his two-phase solver [7].

Suppose we can estimate, for any scramble, how many moves it needs. The estimate is never larger than the truth, and one move changes it by at most one. Call it h.

Such an estimate cuts the tree. Walk down the tree of moves and keep track of how many moves are left. If the estimate for a position is 20 while only 18 moves remain, that whole branch can be dropped: it cannot reach the solved cube in time. Figure 6 shows this. Below every position the table is read, and a branch whose estimate is larger than the moves still available is dropped without being explored. The estimate may be too small, which only means less cutting. It may never be too large.

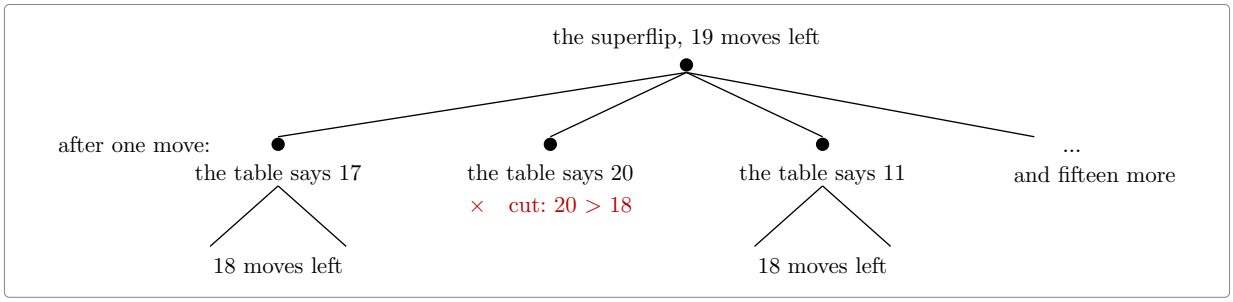


Figure 6: The search, and its scissors.

3.2 The origin of the estimate

The idea is to forget most of the cube. Keep only part of the information, say how the corners are twisted and where the four middle-layer edges sit, and call what is left a **summary**. Many scrambles share a summary. Moves act on summaries as well as on cubes, and there are few enough summaries that a computer can work out, once and for all, the exact distance from the solved summary to every other one. That table of distances is the estimate. A scramble needs at least as many moves as its summary does.

A summary is read straight off the sticker numbers. Figure 7 shows the three questions it asks. A corner has one sticker belonging to the up or down face, and that sticker sits in one of three places, which is 0, 1 or 2. An edge is either the right way round or turned over, which is 0 or 1. The four edges of the middle layer occupy four of the twelve edge slots. The figure shades them on the two visible faces, where three of the four can be seen. Nothing else about the position is recorded.

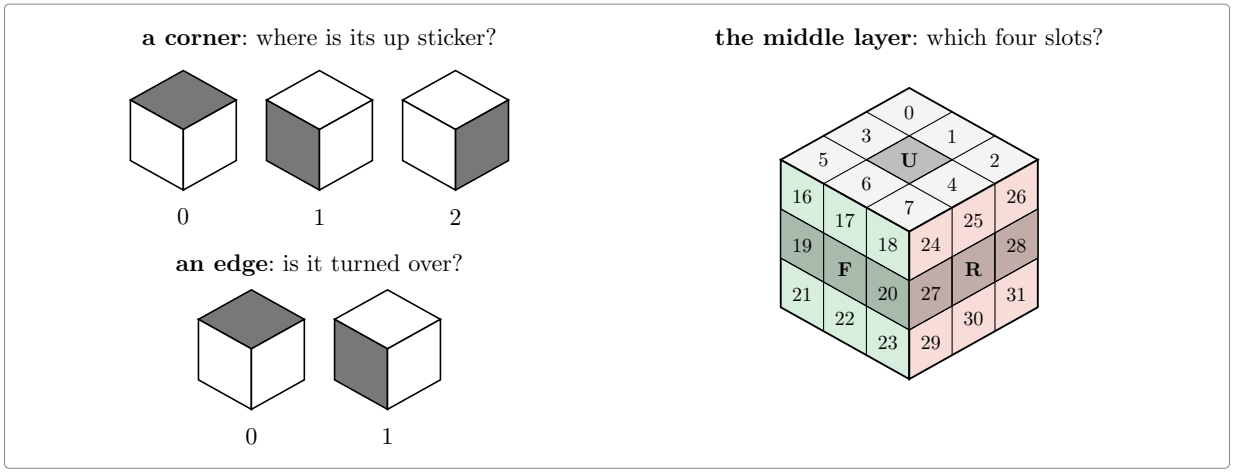


Figure 7: The three questions a summary asks.

The summary is the product of the three answers:

summary	values	
how the eight corners are twisted	2 187	$= 3^7$
how the twelve edges are flipped	2 048	$= 2^{11}$
where the four middle-layer edges sit	495	4 places among 12
the phase 1 summary, all three together	**2 217 093 120**	

The name comes from Kociemba's solving algorithm, which works in two phases. Its first phase brings exactly these three answers to zero, and the summary is what it watches while doing so. The algorithm itself plays no part here. We borrow only the name, because it is the one everybody uses for this table.

Two billion is small next to 43 quintillion. **Every summary stands for exactly 19 508 428 800 real scrambles**, and that division comes out even. The table records, for each summary, its distance from the solved summary. No entry is larger than 12, which is how deep the deepest summary lies. So four bits hold one entry, and the whole table is **1.18 GB**.

3.3 A failed search as a proof

The search answers "maybe solvable in d moves" or "no". The **no** is the side to trust. It means every branch was cut, either because the depth ran out or because the table said so. So a no to "is the superflip solvable in 19 moves?" is the theorem we want.

The answer does not depend on the table being right. If an entry were too small, the search would cut less and take longer. Only two local properties matter: the solved summary has distance 0, and one move changes the value by at most one. That is what brings the cost down. A table with two billion entries is never proved correct. It is only checked.

4 The search in Rocq

The general principle is one file of about a hundred lines, `Search.v`, and it does not mention the cube. It takes a group, a set of moves, an estimate h, and the two assumptions:

```
Hypothesis h1    : h 1 = 0.
Hypothesis hstep : forall g m, m \in S -> h g <= (h (g * m)).+1.

Fixpoint search (d : nat) (g : gT) : bool :=
  (h g <= d) &&
  ((g == 1) ||
   (if d is d'.+1 then has (fun m => search d' (g * m)) Sseq else false)).

Corollary searchN d g : search d g = false -> g \notin ball S d.
```

Again word by word:

- `gT` is the group the file works in. It is a variable, so nothing here is about the cube; the cube is what it gets instantiated with later.
- `1` is the unit of that group, which for the cube is the **solved** position. So `h 1 = 0` says the estimate of a solved cube is zero, and `g == 1` asks whether the search has arrived.
- `Sseq` is the list of moves, the eighteen of them, in the order the search walks over them. `S` is the same thing seen as a set.
- `g * m` is the position `g` followed by the move `m`. Mathcomp applies permutations on the right, so `(g * m) f` is `m (g f)`. The product reads left to right, like a sequence of moves, and not right to left like a composition of functions.
- `h g <= d` is the cut, and `(h (g * m)).+1` is the estimate after a move plus one, which is the assumption that one move changes the estimate by at most one.
- `has (fun m => search d' (g * m)) Sseq` tries every move with one fewer move available, and answers as soon as one of them succeeds.

The last line is the contract of the whole development: **if the search returns false, the position is not within d moves.**

That is a theorem, proved once, about any estimate that satisfies the two assumptions. Everything else in the development is there for one of two reasons: to discharge those two assumptions for the real table, or to make the search fast enough to run.

What the table has to satisfy, and what it does not. `Coord.v` asks for three things and checks two:

```
Variable coord : {perm facelet} -> X.
Variable act   : X -> {perm facelet} -> X.
```

```
Hypothesis coordM : forall g m, coord (g * m) = act (coord g) m.

Variable D : X -> nat.
Hypothesis D0    : D (coord 1) = 0.
Hypothesis Dstep : forall x m, m \in Sset -> D x <= (D (act x m)).+1.
```

`coord` is the summary of a position, and `X` is whatever the summaries are. `act` is how a move acts on a summary directly, without going back to the position it came from. For the phase 1 summary it is two lookups in a move table. `coordM` is the one thing proved about the summary itself, and it is what makes the pair worth having. It says that summarising after a move gives the same answer as acting on the summary. So the search never computes a summary from a position. It carries the summary beside the position and brings it up to date one move at a time, at the cost of a lookup. It still carries the position, because the summary cannot say whether the cube is solved and the position can.

Beside the `search` of `Search.v`, the search that runs has this shape. Names are simplified and the machine-integer details left out. The real one is `searchz3` in `Farp1.v`:

```
Fixpoint search (d : nat) (g : gT) (x : summary) (p : move) : bool :=
  (D x <= d) &&
  ((g == 1) ||
   (if d is d'.+1
    then has (fun m => search d' (g * m) (act x m) m) (allowed p)
    else false)).
```

Two things travel down the tree instead of one, and each is used for exactly one job:

- `D x <= d` is the cut. It reads the table at the summary `x`, and never looks at the position.
- `g == 1` asks whether the cube is solved. It reads the position `g`, and never looks at the summary.
- `g * m` moves the position and `act x m` moves the summary, side by side, one move at a time. That step is `coordM` being used, and it is why the summary never has to be recomputed from the position.
- `p` is the move just played, and `allowed p` is the list of moves the rules permit after it. That is where redundant sequences are dropped.

The two conditions. `D` is the estimate. It takes a summary and gives back a number, and that number is read from the table. `D0` and `Dstep` are everything asked of it. `D0` says the solved cube gets zero. `Dstep` says that one move lowers the estimate by at most one.

That is a weak demand. The table is never proved to hold the true distance to the solved cube. A table of zeros passes both conditions. It would prune nothing and the search would run for ever, but it would not make the search give a wrong answer.

How the two conditions are checked. Look again at `Dstep`. Unlike `D0`, it does not mention `coord`. It speaks of every value `x` in `X`, and not only of the values that are the summary of a real position. That is deliberate, and it is what makes the check possible. `X` is a finite set of 2.2 billion values, so the check runs over all of them and never has to know which come from a cube. `D0` is then one lookup, and `Dstep` is one sweep: 2.2 billion summaries, eighteen moves each, one comparison apiece. Those sweeps are what the *certificate* files do: `FsmChk.v`, `FsrChk.v`, `SlrChk.v`, `P1TsChk.v` and `Farp1chk.v`, listed again at the end of this note, each ending in its own `Qed`.

Nothing here asks where the table came from. Any program in any language can write it. Ours is an OCaml generator that writes it out as Rocq source, and the two conditions are checked on it afterwards.

Three cuts at the top of the tree. They are three different arguments and we keep them apart. The first two apply once each, to the first move and to the second. The third applies at every move from the third on.

The first move: eighteen become two. The superflip looks the same from every angle. There are 48 ways of putting a cube back into the space it came from: any of the six faces can be turned to the top, each in four positions, which makes twenty-four, and each of those seen in a mirror as well. Relabelling the superflip's stickers by any of the 48 gives the superflip back. So we may take the first move to be U or U^2 and leave the other sixteen untried.

The second move: eighteen become fifteen. The three that turn the top face again are dropped. Turning the top face twice running merges into a single turn, so those give a word of nineteen moves or fewer, and the proof covers shorter words at a smaller depth rather than by a search of their own.

The three that turn the **bottom** face look just as droppable, and they are not. Here is why, because this is the case everyone gets wrong. The two turns commute, so UD can be rewritten DU. But symmetry has already pinned the first move to the top face, and turning the cube over to bring that D back to the top gives UD again. It is a fixed point of both rewritings, so neither one removes it. Reid's proof meets the same case and pays for it elsewhere. He keeps the bottom-face second move, and cuts his **third** move instead, using the symmetries that fix the pair of opposite faces.

Our own OCaml program had this wrong. It dropped the three bottom-face second moves, so it searched 24 prefixes where it had to search 30. Nothing about the program looked wrong. It ran for hours, it exhausted its tree, and it reported that no solution of length 19 exists, which is the answer we expected. The error came out only when the same reduction had to be proved in Rocq, and the proof of the bottom-face case could not be written. This is the argument for proving a search rather than trusting it. A cut that is too greedy does not make a search fail. It makes it faster, and it makes it agree with you.

From the third move on: eighteen become fifteen or twelve. This one is not about the superflip and not about the top of the tree. It is one rule, applied at every node. A shortest word never turns the same face twice running, because the two turns merge into one. Of two opposite faces it never uses both orders, because UD and DU give the same position, so fixing one order loses nothing. At a node whose last move was on the top, right or front face, both halves of the rule bite and **twelve** moves are left. At a node whose last move was on the bottom, left or back face, only the first half bites, because the opposite pair has already been cut in the other order, and **fifteen** are left.

Applying that rule to the third move is not an extra assumption. The word after the second move is reduced like any other, and the proof already knew it. The search had been throwing the fact away and trying all eighteen.

None of the three is obvious, and they interact. The bottom-face second move survives only because the rule used from the third move on would otherwise cut the same pair of opposite faces twice, once by symmetry and once by the order convention. Arguments of that shape are easy to get wrong and hard to test. Get one wrong and the search is faster and the answer looks the same. This is what Rocq is for here. Every cut has to be proved before the search may use it. A cut that does not hold is refused rather than rewarded, and we can take cuts that would otherwise be too delicate to trust.

So the depth-19 search becomes $2 \times 15 = 30$ searches of depth 17. The 30 are packed into **seventeen files**, `Runp1_03.v` to `Runp1_17.v`, one per second move and numbered by it, each holding both first moves.

Two of them run far longer than the rest. They are the two whose second move turns the bottom face, D and D^{-1}, where the rule above leaves fifteen branches and not twelve. We cut each in two, one first move to a file: `Runp1_09a.v` and `Runp1_09b.v` in place of a single `Runp1_09.v`, and `Runp1_11a.v` and `Runp1_11b.v` in place of `Runp1_11.v`. Fifteen second moves, two of them split, makes seventeen files. Without the split, the rest of the run would finish long before those two files and would wait for them.

That is how the work is spread over the cores of a machine: seventeen files, seventeen `Qed`s, nothing shared.

5 The optimisations

The first version worked and was far too slow. Getting it from "runs" to "finishes" took a series of changes. We measured each one before and after.

Counting in unary is the enemy. The numbers used by the mathcomp library are Peano numbers: 5 is literally the successor of the successor of the successor of the successor of the successor of zero. So adding n costs n steps, and comparing costs as much again. Rocq also offers machine integers, 63 bits wide with the missing bit going to the garbage collector, and **persistent arrays** of them [8]. Both cost what the hardware costs, and the table is stored that way, fifteen of its four-bit entries to a machine integer. Measured here: a machine-integer operation takes about 0.05 microseconds and a Peano-number operation about 1 microsecond. Converting between the two costs about 0.07 microseconds *per unit*, so converting the number 495 costs 36 microseconds on its own. We rewrote the inner loop so that indices, table values and the depth comparison never leave machine integers:

operation	before	after
reading the summary of a position	32.0 µs	**0.10 µs**
applying a move to a summary	4.60 µs	**0.12 µs**
reading one entry of the packed table	2.99 µs	**0.13 µs**

`and` and `or` are function calls. Rocq evaluates both sides of a boolean test before combining them. So a guard written "either the value is out of range, or the expensive check holds" runs the expensive check on *every* value. Written as a nested `if`, it runs only on the values that get past the guard. Two otherwise identical certificates, one of each shape: **719.7 s against about 80 s**. The same mistake in the search cost another factor of 27.8 on one guard. Every guard in the development is a nested `if` now.

The search itself, twelve times faster. None of the changes make sense until one knows what the search of `Farp1.v` carries at each position. Four things:

- `a`, the position itself as a 48-entry array saying where each sticker went. It has one use, to ask "is this the solved cube?", and each branch gets a fresh one by composing `a` with the move's own array.
- `x`, the three views: for each of the three, the pair of numbers that is its summary, the corner twist and the flip-and-slice index. A move takes each pair to another pair by two lookups in a move table.
- `p`, which face was turned last, from which the list of moves worth trying next is read off.
- `d`, how many moves are left.

The estimate is the largest of **nine** lookups: three tables, at each of the three views. `Fast.v` holds a chain of seven versions, each proved equal to the one before, so no trust is transferred. Measured on one piece at depth 14:

version	seconds	what it removes
the original	70.0	
2	39.1	Peano arithmetic inside the loop: move indices, the depth test and the list of allowed moves all become machine integers, computed once
3	16.5	building the child's 48-entry array before looking at the estimate, when the estimate then rejects the child
4	13.4	comparing all 48 entries against the solved position when the first difference already settles it

version	seconds	what it removes
5	9.7	the last of the nine lookups once one of them already exceeds the moves left
6	8.0	computing the summaries of all three views when the first view already cuts
7	**5.9**	keeping `a` up to date at every position: the list of moves is carried instead, and the position rebuilt from it only for the solved test
	11.9x	

Four of those six steps are the same mistake twice over: work done that a lazier evaluator would have skipped.

Three views of the same position. Rotating the whole cube about a corner axis gives the same position seen differently, and its summary is then a different entry of the same table. So each of the three views gives a lower bound on the number of moves left, and the largest of them is a lower bound too. It is never smaller than any single view gives. That is three times the lookups at each position, in exchange for a sharper cut and a smaller tree. Cube solvers do this as a matter of course, Kociemba's included. What is new here is that the three views are proved legitimate.

How the table is written down costs more than the table. The phase 1 table is emitted as Rocq source, one file per block of 2 097 152 entries, 71 of them. Written as a list, a block is a term: two million nested applications of the list constructor, each holding a machine integer. The `.vo` stores that term and `Require` loads it, and it is far larger than the 17 MB of data in it. Written as an array literal, that is, as a definition whose body has already been evaluated to a primitive array, the `.vo` holds one compact block of memory. Measured on the same 2 097 152 entries:

the block, written as	its `.vo`	loaded
a list	37.8 MB	877 MB
an array literal	**6.0 MB**	**281 MB**

Over all 71 blocks that is 21.5 GB against **5 GB**. It is what let nine workers run in parallel on a 62 GB machine where two had run before.

Folding the table by symmetry. The summary is built around the up-down axis. The twist counts where each corner's up-or-down sticker sits, and the slice says where the four edges between the top and bottom faces are. A symmetry that leaves that axis in place turns a summary into another summary. One that tips the cube onto another axis does not act on these summaries at all. Sixteen of the 48 keep the axis, and they sort the 1 013 760 flip-and-slice values into **64 430 families**, a factor of **15.73**. Two values in the same family are the same distance from solved, so one entry per family is enough. A lookup first replaces the value by its family's representative, carrying the twist through the same symmetry, then reads a table 15.73 times smaller.

That is a different use of symmetry from the three views above, and we keep the two apart. The three views ask the **same** table three questions and keep the largest answer, which sharpens the estimate and cuts the tree. The fold asks the **same** question of a **smaller** table, and the estimate does not change. Symmetry-reduced tables of this kind are standard in cube solvers. What the development adds is a proof that the folded table still satisfies the two conditions.

This is where the weakness of the two conditions pays. The proof nowhere says that the folded table holds distances. It says the table passes `D0` and `Dstep`, and the search needs nothing more. Had the conditions demanded true distances, we would have had to show that the fold preserves them. That is a harder statement about the sixteen symmetries and about what sharing an entry between two summaries does. We never face it. The check is run on the folded table just as it was on the flat one, and it is the same check.

Demanding more would not have cost more either. The same single sweep recognises a table of true distances: it holds them exactly when it passes the two conditions and, at every summary but the solved one, some move lowers the value by exactly one. Asking for less does not buy a cheaper check. It buys the freedom to hand the search a table like this one.

The fold costs the search 1.61 times at depth 16, and it pays everywhere else. A search worker drops from 4.15 GB to **0.85 GB**, so all the pieces run at once instead of in two waves, and checking the table drops from about 5.4 processor hours to **1.35**.

What remains. The same search written in OCaml is about three times faster. We ran both at radius 19 on the reference machine. The OCaml program visits 146 065 078 152 positions in 26.4 processor-hours, which is 0.65 microseconds a position. Rocq takes 87.6 processor-hours over the same tree, which is 2.16. A factor of **3.3**.

We do not assume that the two walk the same tree. Dividing each of the seventeen Rocq pieces by the positions its OCaml counterpart visited gives between 1.98 and 2.52 microseconds, across pieces that differ in size by a factor of two. A Rocq search cutting differently anywhere would show up as scatter there, and there is none.

So the run takes a night because the tree has 146 billion nodes in it, and not because the prover is slow. In OCaml the same tree still costs 26 processor-hours.

6 The files

The proof of the twenty is forty-six hand-written Rocq files. Here is what each group does. The sources carry their own `README.md`, which lists the same files, the scripts beside them, and how to run the whole thing.

the cube, as mathematics	
`Cyc.v`	cyclic permutations, built from the list of points they move
`Rubik333.v`	facelets, the six faces, the eighteen moves, the cube group
`Sym.v`, `Sym16.v`	the 48 symmetries of the cube; the 16 used by the fold
`Ball.v`	balls of radius d, and what "the diameter is at most d" means
`Diameter.v`	the superflip, its 20-move sequence, and what the upper bound would need

the search, in the abstract	
`Search.v`	the search and its contract: a false answer is a proof
`Coord.v`	any summary plus any checked table gives a legal estimate
`Root.v`	the first move, up to symmetry: U or U^2
`Searchr.v`, `Redun.v`	the rules that forbid redundant move sequences

data structures	
`Table.v`	permutations written as the table of their images
`Tabi.v`	the same on machine integers and arrays, and the bridge between
`ssrint63.v`	the machine-integer toolbox used throughout

the summaries and their tables	
`Coordfs.v`, `Coordfsi.v`	the edge-flip and slice summary, packed into 24 bits
`Fstab.v`, `FsTable.v`, `Fsparity.v`	its table and the checks it must pass

the summaries and their tables	
`Phase1.v`	the phase 1 estimate, its table and its certificate, 2215 lines
`Moves.v`	the eighteen moves and the superflip, as tables

the search on the real data	
`Farp1.v`	the three viewing angles and the search built on them, 1404 lines
`Far.v`	the assembly: the superflip is not within d moves, 1124 lines
`Fast.v`, `FastP.v`	the fast search, and the proof that it is the same search
`Runp1_03.v` .. `_17.v`	the seventeen pieces, written by a script
`Farp1main.v`	the theorem over *any* table: 8 seconds, and no data at all
`Farp1inst.v`	the same at the real table and the seventeen real runs
`Diam20.v`	and hence God's number is at least 20

The **certificates** are the files that run the checks, each with its own `Qed`. Four of them cover the move and distance tables and the second summary table: `FsmChk.v`, `FsrChk.v`, `SlrChk.v` and `P1TsChk.v`. `Farp1chk.v` checks the shape of the main table. Two more check the main table itself, split sixteen ways, and the folded table is checked by the `Fold` group, whose slices are `FoldOrbit_00.v` to `FoldOrbit_26.v`. Three of the files that hold the actual numbers are deliberately kept out of the project file, since they do not exist until the generator has run.

Outside Rocq, `ocaml/rubik_par.ml` and `ocaml/rubik_lb.ml` are the reference implementation. The Rocq search reproduces their node counts exactly, which is how we catch a disagreement early, and `bench/p1gen.ml` generates the tables. The `bench/` directory also keeps the experiments that settled design questions, each with its measurements.

7 The development in figures

hand-written Rocq, about the cube	**45 files, 12 725 lines**
`ssrint63.v`, a general int63 toolbox	1 308 lines
generated table sources, in the repository	about 156 000 lines
generated table sources, too big to store	about 165 MB of literals
OCaml reference programs and generators	2 749 lines
build and run scripts	1 031 lines

Positions visited by the radius-19 search, counted by the OCaml program. The Rocq search walks the same tree, and reproduces these counts exactly at the smaller depths where counting it is cheap:

	positions
the smallest piece, `Runp1_11b`	5 575 767 076
the largest piece, `Runp1_10`	10 554 835 820
all seventeen	**146 065 078 152**

The tree grows by 12.22 from one level to the next, measured between depths 17 and 19.

Building the tables costs the same whatever radius is searched afterwards. Measured end to end from a clean tree on the reference machine, a dual-socket Xeon with 62 GB and twelve physical cores:

	wall clock	**processor time**
emitting the tables and compiling them to native code	17 min 21	52 min 13
the coordinate and summary tables	22 min 46	21 min 15
the search and the estimate, over no data at all	11 min 02	30 min 45

	wall clock	processor time
the four certificates for the move and distance tables	56 s	2 min 27
the fold: twelve checks and twenty-seven slices	9 min 52	47 min 57
the shape check against the dummy table	18 s	16 s
in total	**1 h 02**	**2 h 35**

That table says two things. The second line is **serial**, 21 minutes of processor time inside 23 minutes of wall clock, so it is the longest stage by the clock and no number of cores shortens it. The first and fifth lines together are 100 of the 155 processor-minutes, and both are mostly the OCaml compiler turning a table into native code.

8 The theorem and its cost

The statement proved at the top of the chain is

```
Theorem superflip_p1far_real : superflip \notin ball Sset p1depth.
```

In words, the superflip is not within `p1depth` moves of the solved cube, where one script sets the depth before the run. It has **no hypotheses left**. The six computations it rests on are the two summary tables, the three move and distance tables and the searches, and each lives in its own file behind its own `Qed`. Asking Rocq what the proof assumes reports only the primitives of its machine-integer and array interface. Nothing in the chain is admitted.

At depth 19 this says that the superflip cannot be solved in 19 moves. Two lines in `Diam20.v` then turn it into **God's number $\geq$ 20**. That file is where the two ends of the development meet, the cube defined at the bottom of the chain and the search sitting at the top, and it first checks that the searches really were run at 19.

We measured the run twice on the reference machine, once before the fold and the two reductions and once after. It is the same theorem both times.

radius 19, search depth 17	before	after
pieces	18	**17**
workers	9	**18**
memory per worker	4.15 GB	**0.85 GB**
wall clock	11 h 13	**6 h 36**
processor time	85 h 11	**87 h 36**

The pieces do not all take the same time. The shortest took 3 h 38 and the longest 6 h 36, read from the times at which they finished. The run ends when the longest piece ends, so the other sixteen workers are idle before that. That is why two of the fifteen search positions, the ninth and the eleventh, are cut in half. Uncut, each would take about 9 h 50 and the whole run would take that long. Shared evenly over eighteen workers the run would take 4 h 54. So about 1 h 40 goes to idle workers, and saving it means cutting the eight longest pieces in half as well.

The two columns say that the fold and the two reductions cut the wall clock by 41% and left the processor time as it was, 3% higher. The gain in wall clock comes from the memory. A worker needs 0.85 GB, so seventeen pieces fit at once where 4.15 GB allowed only nine. The processor time is a surprise. On the small test we used while writing the reductions, the fold was 1.61 times slower and the reductions 1.86 times faster. Together that is a saving of about a sixth, and the run shows none. We do not trust that small test: it subtracts two large numbers, 245 processor-seconds of table loading from 391 for the whole run, to get 146 of search. The run above is the number to trust for the cost of the theorem. Why the two factors do not add up we have not measured.

Memory. A worker holds the loaded table and nothing else that grows. The search walks down and back up, so it keeps only the current sequence of moves. The 4.15 GB is identical to three decimals

across the nine workers and the same at every depth. The fold brings it to 0.85 and lets all seventeen pieces run at once.

What is left. The proof that God's number is at least 20 costs 87 processor-hours and one night. That is a measured cost, not an estimate. We can see two savings and neither is large. The first is the 1 h 40 of idle workers at the end of the run, which is a matter of how the work is shared out and not of mathematics. The second is the factor of 3.3 against OCaml, and winning all of it would still leave 26 processor-hours. The cost is the tree, and the tree is 146 billion positions. We know of nothing else in the chain that is wasteful.

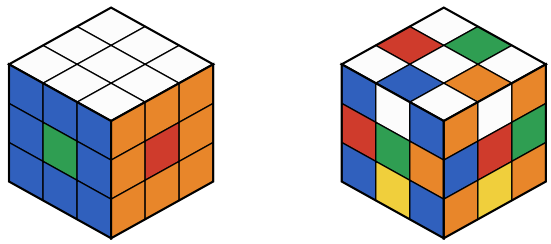
9 Counting in quarter turns

Counted in quarter turns there are twelve moves: the six faces one way and the same six back. A half turn is two moves. The answer in that count is **26** (cube20.org). We prove the lower half: one position cannot be solved in 25 quarter turns.

9.1 The position, and 25 down to 24

The superflip is 24 quarter turns from solved, so it is not far enough away. In August 1998 Reid posted a better position to the Cube-Lovers list [9]: the **four-spot** with the superflip composed onto it. That post is the source of this section, and it is transcribed beside this note.

The four-spot exchanges the front and back colours and the left and right colours. The centres cannot move, so each of those four faces keeps its own colour in one square, which is the spot the pattern is named after. The superflip then turns every edge over.



the four-spot and with the superflip on it

Figure 8: The four-spot, and Reid's position.

Reid's position is 26 quarter turns from solved. His word for it is

$$U^2\ D^2\ L\ F^2\ U^{-1}\ D\ R^2\ B\ U^{-1}\ D^{-1}\ R\ L\ F^2\ R\ U\ D^{-1}\ R^{-1}\ L\ U\ F^{-1}\ B^{-1}$$

That is 21 face turns, five of them half turns. We check it by multiplying out both sides and comparing two lists of 48 places.

Ruling out 25 is ruling out 24. A quarter turn is five four-cycles of the 48 stickers, so it is odd, and a manoeuvre of odd length gives an odd position. Reid's position is even. So every manoeuvre for it has even length, and the searches stop at 24.

9.2 Reid's six beginnings

The argument has two halves and only the second is a computation.

The first is Reid's Proposition 2: a manoeuvre that cannot be shortened can be rewritten, at the same length, to begin with one of six sequences.

$$\begin{array}{cccccc} U & R & F & D & L & B \\ U^2 & R^2 & F^2 & D^2 & L^2 & B^2 \\ U^{-1} & R^{-1} & F^{-1} & D^{-1} & L^{-1} & B^{-1} \end{array}$$

The rewriting uses three operations that change neither the product nor the length: conjugation by one of the sixteen symmetries that fix the position, inversion, and cyclic shift with the letters that move to the end relabelled. The last is why we chose this position. `HProp2.v` is that argument in Rocq, and `HBridge.v` carries it to the orientation the search uses.

Reid does not state one hypothesis his proof needs: the manoeuvre cannot be shortened. Without it the third turn may cancel the second. The Rocq statement carries the hypothesis at no cost, since a shortest manoeuvre is what we want anyway.

Six searches follow. The first beginning is two turns long and is searched 22 further, the other five are three long and are searched 21 further. Each reaches 24 turns.

9.3 The summary, and its table

The estimate is built as before, by keeping a summary of the cube:

summary	**values**	
where the four middle-layer edges sit, each of them the right way round or not	190 080	$= 24 \cdot 22 \cdot 20 \cdot 18$
which four corner places hold the four top corners	70	4 places among 8
how the eight corners are twisted	2 187	$= 3^7$
the three together	**29 099 347 200**	

The 24, 22, 20 and 18 fall by two each time because a place taken by an edge is taken whichever way round that edge is. These summaries are the cosets of Reid's H, which is where the H at the front of the file names comes from.

The table holds the distance from solved of each of the 29 billion summaries. How many lie at each distance agrees with the column Reid published in 1998, and we run that check first. The table is then folded. The sixteen symmetries that keep the up-down axis sort the 190 080 edge values into 12 094 families, a factor of 15.72, and one entry is kept per family. That is 883 MB, measured at 3.86 GB once loaded into the prover.

9.4 The files

Reid's argument	
`HCoord.v`	the three coordinates of a position, on facelet tables
`HRoot.v`	Reid's six positions, and the three views of them
`HProp2.v`	manoeuvres as words, and the three rewritings of them
`HReid.v`	what Proposition 2 rests on
`HBridge.v`	from Proposition 2 to the position the run searches

the search	
`HSearch.v`	the quarter-turn search over Reid's table
`HCanon.v`	the rule the search plays by loses no manoeuvre
`HCut.v`	a cut throws no manoeuvre away
`HPrefix.v`	playing a word is stepping the state
`HPok.v`	the positions the search meets, and the triples it carries
`HRunS.v`, `HSound.v`	a search that fails is a proof that no word exists

the tables, and the assembly	
`HChk.v`	the move tables against the coordinates
`HEdge.v`, `HCorner.v`	a turn acts on the datum, for edges and for corners
`HAgree.v`	the coordinates agree with the tables, everywhere

the tables, and the assembly	
`HSweepC.v`	the three sweeps over the move tables, in six slices
`HSweep.v`	the sweep over the distance table, cut into twelve jobs
`HAdmis.v`	what that sweep buys: the estimate is never too big
`HGlue.v`, `HBound.v`	what the run has to give, and what it gives
`HFinal.v`, `HAll.v`	the bound assembled, and `qdiam25`

9.5 The theorem, and its cost

```
Theorem qdiam25 : ~ diam_le Sq 25.
```

`Sq` is the set of the twelve quarter turns, and `diam_le Sq 25` says every position is within 25 of them. The line says it is not. Reid's position is the witness, and his word puts it at 26. Rocq reports only the primitives of its machine-integer and array interface.

	wall clock	**processor time**
building the table, in OCaml	9 min 50	1 h 43
the same table as fifty-nine Rocq files	3 h 13	6 h 50
the three sweeps over the move tables		1 min 20
the sweep over the distance table, twelve jobs	2 h 51	31 h 42
the six searches, seventy-two pieces, twelve workers	4 h 00	45 h 54
the whole chain in Rocq	**10 h 32**	**87 h 29**

The last row is not the sum of the others. It is the whole chain measured end to end from a directory where nothing is built. We build the OCaml table on the first row once by hand, and it is not part of that run.

The sweeps and the searches are nearly nine tenths of the cost. Checking the table rather than trusting it costs about two thirds of what the searches cost. The quarter-turn work adds eighteen hand-written Rocq files and 6 008 lines.

10 One coset of the upper half

Everything above is a lower bound, and a lower bound rests on one position and one search that finds nothing: the superflip for the twenty face turns, the four-spot with the superflip on it for the twenty-six quarter turns.

The other half of God's number, that twenty moves always suffice, is the huge one, and we do not prove it here. `Diameter.v` holds the reduction for it, and that reduction rests on one assumption.

The published proofs do not solve all 43 quintillion positions one at a time. They cut the cube group into the 2 217 093 120 cosets of a subgroup and solve a whole coset at once. One search settles every one of the 19 508 428 800 positions in it. The positions in a coset are not all the same distance from solved. What the search shows is that none of them is more than 20. A symmetry of the cube carries one coset to another, and the image is solved by the same manoeuvres relabelled, so only one coset per symmetry class is searched. Two things are then needed: that the cosets searched cover every class, and that each search really settles its whole coset.

The first is not a computation, and `Canon.v` proves it. Take as representative the least member of each class, in the order the finite type already carries. The covering property holds because a finite set has a least element. It is eighty lines and assumes nothing.

The second is the computation itself, the one Rokicki, Kociemba, Davidson and Dethridge ran: about a billion seconds of processor time, more than thirty processor years, donated by Google, over 55 882

296 families of cosets. We cannot repeat that. What we can do is one coset, to see what one costs and whether the pieces are in place. This section reports that one coset, the superflip's.

10.1 Cosets

The subgroup is generated by ten of the eighteen moves: the three turns of the top face, the three of the bottom face, and the half turns of the other four. Call them the ten. They generate Reid's H, met above. A coset of H is the set of positions reached by playing the ten from a fixed position. Every position of the cube lies in exactly one coset.

	count
positions of the cube	43 252 003 274 489 856 000
cosets	2 217 093 120
positions in a coset	19 508 428 800
cosets we did	1

We did the superflip's coset. We had the superflip already, from the lower bound, so it is a good enough example of a coset.

10.2 The coset as one map

A position of a coset is named by three numbers: how the eight top and bottom corners sit, how the eight top and bottom edges sit, and how the four middle edges sit. The map holds one bit for each. The arrangements of the corners go in pairs, and a pair shares one word, so the map is 406 425 600 machine words of forty-eight bits, which is 19 508 428 800 bits.

The search starts at the superflip and plays words, setting the bit of every position of the coset it reaches. When every bit is set the theorem follows. Thirty-two positions are not reached. We give each of them a word of twenty moves by hand in `RowWits.v`. We do not trust those words: `RowWitsChk.v` plays each one back and asks for the solved cube.

10.3 The reused and the new

We reuse nearly everything: the search, the map, the tables that step a whole map, the ranking of the three numbers, the replay of the leftover words, the phase one table and the program that generates it, the cube, the permutations and the machine-integer tools.

Four things are new, and none of them is about searching.

- The rank and the sign of a permutation on machine integers. Rocq's library has both, but for permutations that cannot be computed.
- That a position of a coset **is** its three numbers, in both directions. We had one direction only. Without the other the theorem is about triples of numbers and not about the cube.
- The link between the summary the search carries and the position it stands for. The summary is one machine word, the position forty-eight.
- Checks on the tables, one per file, so that each reports separately.

10.4 The unsound stop

Cutting a branch on the table is sound, because a table that says too little only cuts less. Stopping on it is not. Our search used to stop when the table said zero and take the position it had reached as one of the coset. A table of zeros still says too little, so it is still allowed, and it would stop the search everywhere and the theorem would say nothing.

The search now stops on the position. At the bottom it tests the position it is carrying. No part of the proof reads the table, and it costs one comparison.

10.5 Folding the map

The map is 40 320 pages of 20 160 groups, one page for each way the eight top and bottom corners can sit. Sixteen renamings of the cube keep the top and bottom faces in place, send the ten to the ten and leave the superflip alone. Two pages related by a renaming hold the same answer, so one page of each family is enough: 2 768 of the 40 320, a factor of 14.6. A level of the search is one pass over the map, so there is 14.6 times less of it to walk. The price is undoing a renaming whenever a kept page is read.

The kept pages go in pairs of their own, and a pair shares one word as it does on the unfolded side: 1 496 words in place of 2 768. Two hundred and twenty-four of the pages are their own partner and use half a word.

The fold has to be proved as well as written. A renaming sends a member of the coset to a member of the coset. Undoing it gives back the position the page stood for. A map sound after one level is sound after the next. That is the largest single part of the coset's proof.

10.6 The files

the coset and its members	
`Row.v`	a coset, its members, and each member as a bit
`RowMemb.v`, `RowMembi.v`	the cube a member names, and the member a cube gives
`RowMembChk.v`	that bridge, with nothing left open
`RowLeaf.v`, `RowInH.v`	a position of H is its three ranks
`RowInst.v`, `RowReal.v`	the instance: the superflip's own coset

ranking, and the moves	
`Lehmer.v`	the rank and the sign of a permutation, on machine integers
`RowUp8ok.v` .. `RowUp4inv.v`	unranking is a permutation, and undoes the ranking
`RowPar8.v`, `RowPar4.v`, `RowParity.v`	the parity tables, and how a move shifts a parity
`RowPartC.v`, `RowPartM.v`, `RowPartU.v`	each of the three parts is a permutation
`RowMoveH.v`, `RowMoveC.v`, `RowMoveM.v`, `RowMoveU.v`	a move of H is its three halves, each following its table
`RowTab.v`, `RowTabL.v`, `RowTabP.v`, `RowTabF.v`	the tables themselves, and the checks they pass

the map and the search	
`RowMap.v`	the map of a coset, and the pass that steps it
`RowRun.v`	the search, the level loop, and what each owes

the map and the search	
`RowLvl.v`	the pass again, one chunk a page instead of one a word
`RowMask.v`	the folded phase one table, and the moves worth trying
`RowSrch.v`, `RowSrchP.v`	the search with the cuts and the early stop, and its proof
`RowMark.v`	the leftover words marked into the map the run leaves
`RowWits.v`, `RowWitsChk.v`	those words, and the replay that checks them
`RowFinal.v`	every member of the coset is within twenty moves

the fold	
`RowFold.v`	the map folded by the sixteen renamings
`RowFoldSym.v`, `RowFoldConj.v`	the fold tables are the renamings, and they conjugate
`RowFoldPart.v`, `RowFoldSrc.v`, `RowFoldGath.v`	a page renamed, then moved
`RowFoldWrite.v`, `RowFoldLvl.v`	what one write costs, and that a level keeps the map sound
`RowFoldMem.v`, `RowFoldOk.v`	two members that fold together stand or fall together
`RowFoldTot.v`, `RowFoldPorb.v`	the fold tables land in range at every index
`RowFoldEmpty.v`, `RowFoldFinal.v`	the map the run starts from, and the one it leaves
`RowFoldRun.v`, `RowFoldSrch.v`	the folded search and the folded run are sound
`RowFoldSrchI.v`, `RowFoldSrchIP.v`	the same search with the depth as an int, and the two are equal

the two runs	
`RowCub.v`, `RowCubi.v`, `RowCubInst.v`	a position as twenty cubies, carried through the search
`RowCubDef.v`, `RowFoldCubDef.v`, `RowFoldCubDefI.v`	what each run needs, and not one proof
`RowCubBoolI.v`, `RowFoldCubBoolI.v`	the two runs: one boolean each, and nothing else
`RowCubReal.v`, `RowFoldCubReal.v`	the coset on each map, with only that boolean left open
`RowCubProof.v`, `RowCubProofI.v`, `RowFoldCubProof.v`, `RowFoldCubProofI.v`	what a true boolean buys
`RowCubDoneI.v`, `RowFoldCubDoneI.v`	run and proof joined: the two theorems

10.7 The cost

The coset adds sixty-nine hand-written Rocq files and 17 386 lines to the work above, besides the generated tables.

The search ran twice, over the folded map and over the unfolded one, with the same search in both, and both times it filled the map.

the run	wall clock	processor time	peak memory
over the folded map	5 h 53	5 h 51	–
over the unfolded map	7 h 52	7 h 48	23.3 GB

The fold is worth 1.3 times on the wall clock and 1.3 times on processor time. The run does not follow the size of the map, which is 13 times smaller. Most of the work is the search at the deepest levels, and that is the same tree on both sides. What the map sets is the memory. The unfolded map is 3.25 GB against 248 MB, and a level reads one map while it writes the other, so the unfolded run needed 23.3 GB of a 62 GB machine. Told to collect harder, the same run peaks at 13.9 GB and takes four per cent longer.

Two earlier folded runs say where the time went. Over words of twenty-four bits, holding one corner arrangement each, the run took 6 h 00. The same run with the depth left as a unary numeral instead of a machine integer took 8 h 13. Counting in unary is the enemy here too.

The phase one table is 2.9 GB of Rocq source and 4.5 GB once checked, 8 h 48 of processor time. It is generated once and shared with the lower-bound work.

The statement has no hypothesis left.

```
Theorem real_row_superflip_fold_runi m :
  m \in H -> superflip * m \in ball Sset 20.
```

`H` is the group of the ten, and a coset is one of its cosets. Every position of the superflip's coset is within twenty moves. Rocq reports only the primitives of its machine-integer and array interface. The unfolded run proves the same statement from its own map.

11 Conclusion

We prove two lower bounds. No position of the cube is solved in 19 face turns, and none in 25 quarter turns. In each case one position is the witness, the superflip for the first and the four-spot with the superflip on it for the second. In each case the proof is a search that comes back empty over a table of estimated distances.

We do not prove that 20 and 26 always suffice. The upper half is a different computation. It is not one search that finds nothing, but two billion searches that must each find everything. We did one of the two billion, and the section above says what it cost.

The three share a trunk. The cube itself, as permutations of the forty-eight facelets, with the eighteen moves and the group they generate. Balls, and what it means for a position to be within d moves. The abstract search, whose contract is that a false answer is a proof. The rule that any summary of a position, together with any table that passes one check, gives an estimate that is never too big. And the tables themselves, held as machine integers in arrays rather than as lists of unary numbers.

What each of the three needed of its own:

- **The twenty face turns.** The phase one summary, which is the edge flips and the slice, its table, and the certificate that checks the table. Three viewing angles of the same search, and seventeen pieces run side by side.
- **The twenty-six quarter turns.** Reid's Proposition 2, which is the one piece of the development argued by hand rather than computed, and the parity argument that turns 25 into 24. A second and much larger summary, 29 billion values, with a table of its own and a sweep of its own.
- **One coset.** A coset held as a map of bits rather than as a tree of positions. That needs the rank and the sign of a permutation on machine integers, the bijection between a position and its three

ranks, a pass that steps a whole map one move at a time, the fold by the sixteen renamings, and words supplied by hand for the positions the search does not reach.

The whole development, counted in hand-written Rocq and leaving out the generated tables, is 37 898 lines. Each line of the table counts what that piece adds to the ones above it.

	files	**lines**
the superflip, for the twenty face turns	53	14 504
the four-spot, for the twenty-six quarter turns	18	6 008
one coset of the upper bound	69	17 386
in all	**140**	**37 898**

One point is worth recording. Our own OCaml prototype for the first bound dropped six of the thirty beginnings it should have tried. It ran for hours and gave the expected answer. The Rocq proof is what found it. A cut that is too greedy does not make a search fail. It makes it faster, and it makes it agree with you.

This development was written with the help of Claude, Anthropic's coding assistant.

The sources are at `github.com/thery/DoubleCover/code/Rubik`, with the note, its figures and Reid's transcribed post beside them.

References

1. Rokicki, T., Kociemba, H., Davidson, M., Dethridge, J.: The Diameter of the Rubik's Cube Group Is Twenty. SIAM Journal on Discrete Mathematics. 27, 1082–1105 (2013). https://doi.org/10.1137/120867366
2. The Rocq Development Team: The Rocq Prover, https://rocq-prover.org/
3. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo, 2021, https://doi.org/10.5281/zenodo.4457887
4. Korf, R.E.: Depth-first Iterative-deepening: An Optimal Admissible Tree Search. Artificial Intelligence. 27, 97–109 (1985). https://doi.org/10.1016/0004-3702(85)90084-0
5. Culberson, J.C., Schaeffer, J.: Pattern Databases. Computational Intelligence. 14, 318–334 (1998). https://doi.org/10.1111/0824-7935.00065
6. Korf, R.E.: Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In: Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97). pp. 700–705 (1997)
7. Kociemba, H.: The Two-Phase Algorithm, https://kociemba.org/cube.htm
8. Armand, M., Grégoire, B., Spiwack, A., Théry, L.: Extending Coq with Imperative Features and its Application to SAT Verification. In: Interactive Theorem Proving (ITP 2010). pp. 83–98. Springer (2010)
9. Reid, M.: superflip composed with four spot. Post to the Cube-Lovers mailing list, 2 August 1998, The address cited by Rokicki et al. no longer resolves; this is a transcription, https://github.com/thery/DoubleCover/blob/main/doc/reid-1998-fourspot.md